

Funções com parâmetro de acumulação

- A ideia que está na base destas funções é que elas vão ter um parâmetro extra (o **acumulador**) onde a resposta vai sendo construída e gravada à medida que a recursão progride.
- O acumulador vai sendo actualizado e passado como parâmetro nas sucessivas chamadas da função.
- Uma vez que o acumulador vai guardando a resposta da função, **o seu tipo deve ser igual ao tipo do resultado da função**.

Exemplo: A função que inverte uma lista.

A função `inverte` chama uma função auxiliar `inverteAc` com um parâmetro de acumulação e inicializa o acumulador.

```
inverte :: [a] -> [a]
inverte l = inverteAc l []
  where inverteAc [] ac = ac
        inverteAc (x:xs) ac = inverteAc xs (x:ac)
```

O acumulador é inicialmente a lista vazia.

Quando a lista é vazia o acumulador tem a solução completa.

A chamada recursiva é feita actualizando o acumulador.

Funções com parâmetro de acumulação

```
inverte :: [a] -> [a]
inverte l = inverteAc l []
  where inverteAc [] ac = ac
        inverteAc (x:xs) ac = inverteAc xs (x:ac)
```

```
inverte [1,2,3] = inverteAc [1,2,3] []
                = inverteAc [2,3] [1]
                = inverteAc [3] [2,1]
                = inverteAc [] [3,2,1]
                = [3,2,1]
```

A solução está a ser construída no acumulador.

Esta versão é bastante mais eficiente que a função `reverse` anteriormente definida (porque usa o `++` que tem que atravessar a primeira lista).

```
reverse :: [a] -> [a]
reverse [] = []
reverse (x:xs) = reverse xs ++ [x]
```

```
reverse [1,2,3] = (reverse [2,3]) ++ [1]
                = ((reverse [3]) ++ [2]) ++ [1]
                = (((reverse []) ++ [3]) ++ [2]) ++ [1]
                = [] ++ [3] ++ [2] ++ [1]
                = ...
                = [3,2,1]
```

Funções com parâmetro de acumulação

Podemos sistematizar as seguintes regras para definir funções usando esta técnica:

1. Colocar o acumulador como um parâmetro extra.
2. O acumulador deve ser do mesmo tipo que o do resultado da função.
3. Devolver o acumulador no acaso de paragem da função.
4. Actualizar o acumulador na chamada recursiva da função.
5. A função principal (sem acumulador) chama a função com parâmetro de acumulação, inicializando o acumulador.

Exemplo: O somatório de uma lista de números.

```
somatorio :: Num a => [a] -> a
somatorio l = sumAc l 0
  where sumAc :: Num a => [a] -> a -> a
        sumAc [] n = n
        sumAc (x:xs) n = sumAc xs (x+n)
```

```
somatorio [1,2,3]
= sumAc [1,2,3] 0
= sumAc [2,3] (1+0)
= sumAc [3] (2+1+0)
= sumAc [] (3+2+1+0)
= 6
```

Funções com parâmetro de acumulação

Exemplo: O máximo de uma lista não vazia.

```
maximo :: Ord a => [a] -> a
maximo (x:xs) = maxAc xs x
  where maxAc :: Ord a => [a] -> a -> a
        maxAc [] n = n
        maxAc (x:xs) n = if x > n then maxAc xs x
                              else maxAc xs n
```

```
maximo [2,7,3,9,4] = maxAc [7,3,9,4] 2
                  = maxAc [3,9,4] 7
                  = maxAc [9,4] 7
                  = maxAc [4] 9
                  = maxAc [] 9
                  = 9
```

Funções com parâmetro de acumulação

Exemplo: A função factorial.

```
factorial :: Integer -> Integer
factorial n = factAc n 1
  where factAc :: Integer -> Integer -> Integer
        factAc 0 x = x
        factAc n x | n>0 = factAc (n-1) (n*x)
```

```
factorial 5 = factAc 5 1
            = factAc 4 (5*1)
            = factAc 3 (4*5*1)
            = factAc 2 (3*4*5*1)
            = factAc 1 (2*3*4*5*1)
            = factAc 0 (1*2*3*4*5*1)
            = 120
```

Funções com parâmetro de acumulação

Exemplo: A função `stringToInt :: String -> Int` que converte uma string (representando um número inteiro positivo) num valor inteiro.

```
stringToInt "5247" = 5247
```

```
import Data.Char

stringToInt :: String -> Int
stringToInt (x:xs) = aux xs (digitToInt x)
  where aux :: String -> Int -> Int
        aux (h:t) ac = aux t (ac*10 + (digitToInt h))
        aux [] ac = ac
```

```
stringToInt "5247" = aux "247" 5
                  = aux "47" (50+2)
                  = aux "7" (520+4)
                  = aux "" (5240+7)
                  = 5247
```

Listas por compreensão

Na matemática é costume definir conjuntos por compreensão à custa de outros conjuntos.

O conjunto .

O conjunto .

Em Haskell podem definir-se **listas por compreensão**, de modo semelhante, construindo novas listas à custa de outras listas.

```
[ 2*x | x <- [10,3,7,2] ]
```

A lista [20,6,14,4] .

```
[ n | n <- [4,-5,8,20,-7,1] , 0<=n, n<=10 ]
```

A lista [4,8,1] .

Listas por compreensão

A expressão `x <- [1,2,3,4,5]` é chamada de gerador da lista.

A expressão `10 <= x^2` é uma guarda que restringe os valores produzidos pelo gerador que a precede.

```
> [ x^2 | x <- [1,2,3,4,5], 10 <= x^2 ]  
[16,25]
```

As listas por compreensão podem ter vários geradores e várias guardas.

```
> [ (x,y) | x <- [1,2,3], y <- [4,6] ]  
[(1,4), (1,6), (2,4), (2,6), (3,4), (3,6)]
```

Mudar a ordem dos geradores muda a ordem dos elementos na lista final.

```
> [ (x,y) | y <- [4,5], x <- [1,2,3] ]  
[(1,4), (2,4), (3,4), (1,5), (2,5), (3,5)]
```

Um gerador pode depender de variáveis introduzidas por geradores anteriores.

```
> [ (x,y) | x <- [1..3], y <- [x..3] ]  
[(1,1), (1,2), (1,3), (2,2), (2,3), (3,3)]
```


Listas por compreensão

Pode-se usar a notação `..` para representar uma enumeração com o passo indicado pelos dois primeiros elementos. Caso não se indique o segundo elemento, o passo é um.

```
> [1..5]  
[1, 2, 3, 4, 5]
```

```
> [1, 10..100]  
[1, 10, 19, 28, 37, 46, 55, 64, 73, 82, 91, 100]
```

```
> [20, 15..(-7)]  
[20, 15, 10, 5, 0, -5]
```

```
> ['a'..'z']  
"abcdefghijklmnopqrstuvwxyz"
```

Listas infinitas

É possível também definir listas infinitas.

```
> [1..]  
[1,2,3,4,5,6,7,8,9,10,11,...
```

```
> [0,10..]  
[0,10,20,30,40,50,60,70,80,90,100,110,120,130,...
```

```
> [ x^3 | x <- [0..], even x ]  
[0,8,64,216,512,1000,...
```

```
> take 10 [3,3..]  
[3,3,3,3,3,3,3,3,3,3]
```

```
> zip "Haskell" [0..]  
[( 'H',0), ( 'a',1), ( 's',2), ( 'k',3), ( 'e',4), ( 'l',5), ( 'l',6)]
```

Funções e listas por compreensão

Podem-se definir funções usando listas por compreensão.

Exemplo: A função de ordenação de listas *quick sort*.

```
qsort :: (Ord a) => [a] -> [a]
qsort [] = []
qsort (x:xs) = (qsort [y | y<-xs, y<x]) ++ [x] ++ (qsort [y | y<-xs, y>=x])
```

Esta versão do *quick sort* faz duas travessias da lista para fazer a sua partição e, por isso, é pior do que a versão anterior com a função auxiliar *parte*.

Funções e listas por compreensão

Exemplo: Usando a função `zip` e listas por compreensão, podemos definir a função que calcula a lista de posições de um dado valor numa lista.

```
posicoes :: Eq a => a -> [a] -> [Int]
posicoes x l = [ i | (y,i) <- zip l [0..], x == y]
```

O lado esquerdo do gerador da lista é um padrão.

A *lazy evaluation* do Haskell faz com que não seja problemático usar uma lista infinita como argumento da função `zip`.

```
> posicoes 3 [4,5,3,4,5,6,3,5,3,1]
[2,6,8]
```

Funções e listas por compreensão

Exemplo: Calcular os divisores de um número positivo.

```
divisores :: Integer -> [Integer]
divisores n = [ x | x <- [1..n], n `mod` x == 0]
```

Testar se um número é primo.

```
primo :: Integer -> Bool
primo n = divisores n == [1,n]
```

```
> primo 5
True
> primo 1
False
```

Lista de números primos até um dado n.

```
primosAte :: Integer -> [Integer]
primosAte n = [ x | x <- [1..n], primo x]
```

Lista infinita de números primos.

```
primos :: [Integer]
primos = [ x | x <- [2..], primo x]
```

Crivo de Eratóstenes

Um algoritmo mais eficiente para encontrar números primos, é o [Crivo de Eratóstenes](#) (assim chamado em honra ao matemático grego que o inventou), que permite obter todos os números primos até um determinado valor n . A ideia é a seguinte:

- Começa-se com a lista $[2..n]$.
- Guarda-se o primeiro elemento da lista (pois é um número primo) e removem-se da cauda da lista todos os múltiplos desse primeiro elemento.
- Continua-se a aplicar o passo anterior à restante lista, até que a lista de esgote.

```
crivo [] = []  
crivo (x:xs) = x : crivo [ n | n <- xs , n `mod` x /= 0 ]
```

```
primosAte n = crivo [2..n]
```

Lista infinita de números primos.

```
primos = crivo [2..]
```

Factorização em primos

O [Teorema Fundamental da Aritmética](#) (enunciado pela primeira vez por Euclides) diz que qualquer número inteiro (maior do que 1) pode ser decomposto num produto de números primos. Esta decomposição é única a menos de uma permutação.

Exemplo: Com o auxílio da lista de números primos, podemos definir uma função que dado um número (maior do que 1), calcula a lista dos seus factores primos.

```
factoriza :: Integer -> [Integer]
factoriza n = aux n primos
  where aux 1 _ = []
        aux n (x:xs)
          | n `mod` x /= 0 = aux n xs
          | otherwise     = x : aux (n `div` x) (x:xs)
```

```
> factoriza 94753
[19,4987]
> factoriza 9475312
[2,2,2,2,7,11,7691]
```

Funções de ordem superior

Em Haskell, as funções são entidades de [primeira ordem](#). Ou seja,

- As funções podem [receber outras funções como argumento](#).

```
twice :: (a -> a) -> a -> a
twice f x = f (f x)
```

Exemplos:

```
dobro :: Int -> Int
dobro x = x + x
```

```
quadruplo :: Int -> Int
quadruplo x = twice dobro x
```

```
retira2 :: [a] -> [a]
retira2 l = twice tail l
```

```
quadruplo 5 = twice dobro 5
             = dobro (dobro 5)
             = (dobro 5) + (dobro 5)
             = (5+5) + (5+5)
             = 10 + 10
             = 20
```

```
retira2 [4,5,7,0,9] = twice tail [4,5,7,0,9]
                    = tail (tail [4,5,7,0,9])
                    = tail [5,7,0,9]
                    = [7,0,9]
```


Funções de ordem superior

- As funções podem devolver outras funções como resultado.

```
mult :: Int -> Int -> Int
mult x y = x * y
```

O tipo é igual a `Int -> (Int -> Int)`, porque `->` é associativo à direita

Exemplos:

```
triplo :: Int -> Int
triplo = mult 3
```

`triplo` tem o mesmo tipo que `mult 3`

```
triplo 5 = mult 3 5
         = 3 * 5
         = 15
```

`mult 3 5 = (mult 3) 5`, porque a aplicação é associativa à esquerda

```
twice (mult 2) 5 = (mult 2) ((mult 2) 5) = mult 2 (mult 2 5)
                 = 2 * (mult 2 5)
                 = 2 * (2 * 5)
                 = 20
```

map

Consideremos as seguintes funções:

Estas funções fazem coisas distintas entre si, mas **a forma como operam é semelhante**: aplicam uma transformação a cada elemento da lista de entrada.

Dizemos que estas funções têm um **padrão de computação** comum, e apenas diferem na função que é aplicada a cada elemento da lista.

```
triplos :: [Int] -> [Int]
triplos [] = []
triplos (x:xs) = 3*x : triplos xs
```

```
maiusculas :: String -> String
maiusculas [] = []
maiusculas (x:xs) = toUpper x : maiusculas xs
```

```
somapares :: [(Float,Float)] -> [Float]
somapares [] = []
somapares ((a,b):xs) = a+b : somapares xs
```

A função **map** do `Prelude` sintetiza este padrão de computação, abstraindo em relação à função que é aplicada aos elementos da lista.

map

```
map :: (a -> b) -> [a] -> [b]
map f [] = []
map f (x:xs) = (f x) : (map f xs)
```

map é uma função de ordem superior que recebe a função *f* que é aplicada ao longo da lista.

Exemplos:

```
triplos :: [Int] -> [Int]
triplos l = map (3*) l
```

```
maiusculas :: String -> String
maiusculas xs = map toUpper xs
```

```
somapares :: [(Float,Float)] -> [Float]
somapares l = map aux l
  where aux (a,b) = a+b
```

```
triplos [1,2] = map (3*) [1,2]
              = 3*1 : map (3*) [2]
              = 3*1 : 3*2 : map (3*) []
              = 3*1 : 3*2 : []
              = 3:6:[] = [3,6]
```

Usando listas por compreensão, poderíamos definir a função `map` assim:

```
map f l = [ f x | x <- l ]
```

filter

Consideremos as seguintes funções:

Estas funções fazem coisas distintas entre si, mas **a forma como operam é semelhante**: *selecionam da lista de entrada os elementos que verificam uma dada condição.*

Estas funções têm um **padrão de computação** comum, e apenas diferem na condição com que cada elemento da lista é testado.

```
pares :: [Int] -> [Int]
pares [] = []
pares (x:xs) = if even x
                then x : pares xs
                else pares xs
```

```
positivos :: [Double] -> [Double]
positivos [] = []
positivos (x:xs)
    | x > 0      = x : positivos xs
    | otherwise = positivos xs
```

A função **filter** do Prelude sintetiza este padrão de computação, abstraindo em relação à condição com que os elementos da lista são testados.

filter

```
filter :: (a -> Bool) -> [a] -> [a]
filter p [] = []
filter p (x:xs)
  | p x      = x : filter p xs
  | otherwise = filter p xs
```

filter é uma função de ordem superior que recebe a condição *p* (um predicado) com que cada elemento da lista é testado.

Exemplos:

```
pares :: [Int] -> [Int]
pares l = filter even l
```

```
positivos :: [Double] -> [Double]
positivos xs = filter (>0) xs
```

```
pares [1,2,3,4] = filter even [1,2,3,4]
               = filter even [2,3,4]
               = 2 : filter even [3,4]
               = 2 : filter even [4]
               = 2 : 4 : filter even []
               = 2:4:[] = [2,4]
```

Usando listas por compreensão, poderíamos definir a função `filter` assim:

```
filter p l = [ x | x <- l, p x ]
```

Funções anónimas

Em Haskell é possível definir funções sem lhes dar nome, através [expressões lambda](#).

Por exemplo, `\x -> x+x` É uma **função anónima** que recebe um número x e devolve como resultado $x+x$.

```
> (\x -> x+x) 5  
10
```

Uma expressão lambda tem a seguinte forma (a notação é inspirada no [-calculus](#)):

`\padrão ... padrão -> expressão`

Exemplos:

```
> (\x y -> x+y) 3 8  
11
```

```
> (\(x1,y1) (x2,y2) -> (x1+x2,y1+y2)) (3,2) (7,9)  
(10,11)
```

```
> (\(x:xs) -> xs) [1,2,3]  
[2,3]
```

```
> (\(x:xs) y -> y:xs) [1,2,3] 9  
[9,2,3]
```

Funções anónimas

As expressões lambda são úteis para evitar declarações de pequenas funções auxiliares.

Exemplo: Em vez de

```
trocapares :: [(a,b)] -> [(b,a)]
trocapares l = map troca l
  where troca (x,y) = (y,x)
```

pode-se escrever

```
trocapares l = map (\(x,y) -> (y,x)) l
```

Exemplo:

```
multiplosDe :: Int -> [Int] -> [Int]
multiplosDe n xs = filter (\x -> mod x n == 0) xs
```

Funções anónimas

As expressões lambda podem ser usadas na definição de funções. Por exemplo:

```
soma x y = x + y
```

```
soma1 = \x y -> x + y
```

soma, soma1, soma2 e soma3 são funções equivalentes

```
soma2 = \x -> (\y -> x + y)
```

```
soma3 x = \y -> x + y
```

Os operadores infixos aplicados apenas a um argumento (a que se dá o nome de **secções**), são uma forma abreviada de escrever funções anónimas.

Exemplos:

```
(+y)  ≡  \x -> x+y
```

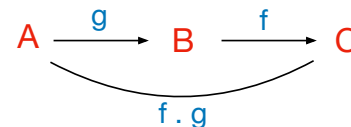
```
(x+)  ≡  \y -> x+y
```

```
(*3) ≡  \x -> x*3
```


Funções de ordem superior

- **(.)** composição de funções

```
(.) :: (b -> c) -> (a -> b) -> a -> c  
(.) f g x = f (g x)
```



Exemplo:

```
ultimo :: [a] -> a  
ultimo = head . reverse
```

```
ultimo [1,2,3] = (head . reverse) [1,2,3]  
               = head (reverse [1,2,3])  
               = head [3,2,1]  
               = 3
```

- **flip** troca a ordem dos argumentos de uma função binária.

```
flip :: (a -> b -> c) -> b -> a -> c  
flip f x y = f y x
```

```
> (^) 3 2  
9  
> flip (^) 3 2  
8
```

Exemplo:

```
mytake :: [a] -> Int -> [a]  
mytake = flip take
```

```
mytake [1..10] 3 = flip take [1..10] 3  
                 = take 3 [1..10]  
                 = [1,2,3]
```

Funções de ordem superior

- **curry** transforma uma função que recebe como argumento um par, numa função equivalente que recebe um argumento de cada vez.

```
curry :: ((a,b) -> c) -> a -> b -> c
curry f x y = f (x,y)
```

- **uncurry** transforma uma função que recebe dois argumentos (um de cada vez), numa função equivalente que recebe um par.

```
uncurry :: (a -> b -> c) -> (a,b) -> c
uncurry f (x,y) = f x y
```

Exemplo:

```
quocientes :: [(Int,Int)] -> [Int]
quocientes l = map (\(x,y) -> div x y) l
```

Ou, em alternativa,

```
quocientes l = map (uncurry div) l
```

```
> quocientes [(10,3), (20,4)]
[3,5]
```